

LeetCode 4059

Lexicographically Largest Power Array

Hard · Weekly Contest 520 (Q4) · Greedy + bit partitioning · C++

1. Problem

You get an array **nums** of length n and may rearrange it into any permutation **perm**. Define an array **power** of length 15: for each i , **power[i]** is the largest j such that the first j elements of **perm** **all have bit $(14 - i)$ set**. So **power[0]** is about bit 14 and **power[14]** is about bit 0. Return the **lexicographically largest** power array you can get.

- Constraints: $1 \leq n \leq 5 \cdot 10^4$, $0 \leq \text{nums}[i] < 2^{15}$ (that is why there are 15 bits).

Input	Output	Why
[7,5]	[0×12, 2,1,2]	perm [7,5]: both have bit 2 → 2; only 7 has bit 1 → 1; both have bit 0 → 2
[3,1,7]	[0×12, 1,2,3]	perm [7,3,1]: bit 2 → 1 (only 7); bit 1 → 2 (7,3); bit 0 → 3 (all)

(0×12 means twelve zeros: every number is below 8, so bits 14...3 are never set.)

2. Intuition

Obvious approach. Try all $n!$ orders and keep the best power array. Impossible for $n = 5 \cdot 10^4$.

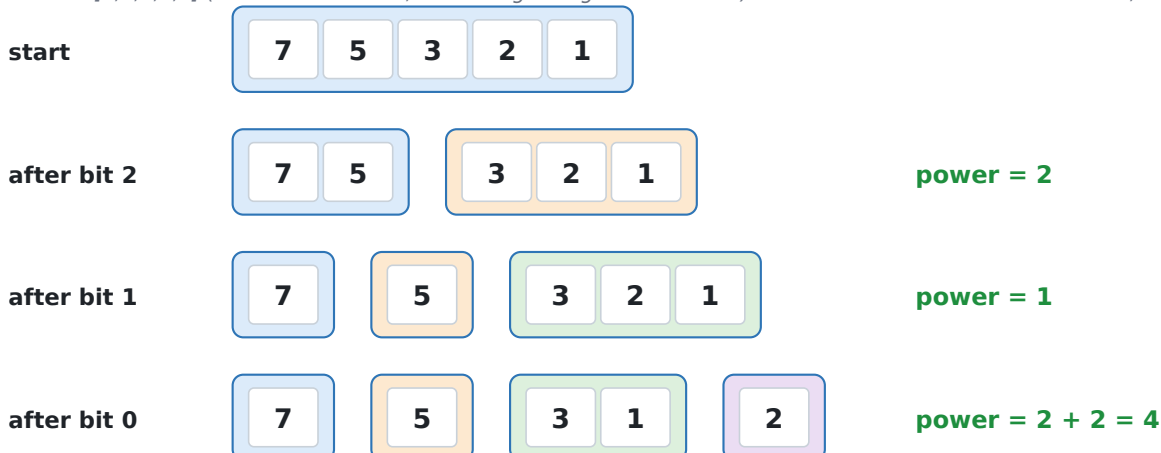
Step 1 — what “lexicographically largest” means here. **power[0]** (bit 14) is the most important entry, then **power[1]** (bit 13), and so on. So we fix the best possible value for bit 14, then the best for bit 13 *without spoiling bit 14*, then bit 12, ... down to bit 0.

Step 2 — how to maximise one bit. **power** for a bit is the length of the leading run of numbers that have the bit. To make it longest, put **all** numbers having the bit in front. For bit 14 say c numbers have it: **power** = c , and to keep **power** = c the first c positions must be *exactly* those numbers. So **perm** now looks like **[numbers with bit 14][numbers without]**, and the order *inside* each group is still completely free. Call each group a **block**.

Step 3 — the next bit. Now the blocks are in a fixed order, each block's inner order is free. To find the longest run for the next bit, walk the blocks from the left:

- a block where **every** number has the bit → the run goes straight through it (its inner order does not matter);
- the **first block that contains a zero** → put its ones first: the run grows by (number of ones) and then it must stop. To reach this maximum that block **must be split** into [ones][zeros];
- blocks after that one cannot change this bit, so leave them alone (still free) for the lower bits.

nums = [7,5,3,2,1] (bits 14...3 are all 0, so nothing changes before bit 2). Each box = a block: inner order free, block order fixed.



Why it works. After each step the set of permutations that keep every higher power at its maximum is *exactly* “blocks in this order, any order inside a block”. Splitting the first mixed block into (ones | zeros) is the only way to reach the maximum run for the current bit, so we never throw away a permutation that is still optimal for the higher bits. This is repeated for all 15 bits.

Why not just sort the array in descending order? Sorting splits every block by every bit, including blocks after the first mixed one. That extra order can shorten a later run. Take [7,5,3,2,1]:

perm	power for bits 2, 1, 0	note
7 5 3 2 1	2, 1, 3	descending sort: bit 0 run stops at 2 (bit 0 of 2 is 0)
7 5 3 1 2	2, 1, 4	the code's answer: [3 2 1] stayed free, so 3 and 1 moved in front of 2

3. Approach (what the code does)

1. `segments` is the ordered list of blocks. It starts as one block holding all of `nums` (lines 6–7).
2. Loop `bit` from 14 down to 0 (line 9). For each bit: `prefix = 0` (run length so far), `stopped = false`, and a new list `nextSegments` (lines 10–12).
3. Go through the blocks in order. If `stopped` already, copy the block unchanged (lines 14–17).
4. Otherwise split the block into `ones` / `zeros` by the current bit (lines 18–22).
5. **No zeros** → the whole block has the bit: `prefix += size`, keep the block (lines 23–25).
6. **Some zeros** → this is the first mixed block: record `power = prefix + ones.size()`, replace the block by `ones` then `zeros`, set `stopped = true` (lines 26–31).
7. If no block was mixed, every number has the bit: `power = prefix (= n)` (line 33). Then `segments = nextSegments` (line 34).

Because bit goes $14 \rightarrow 0$, `power` is filled in exactly in the order $i = 0 \dots 14$, which is the order the answer needs.

4. State / Formula

Name	Meaning
<code>segments</code>	ordered list of blocks; the order of blocks is fixed by the higher bits, the order inside a block is free
<code>prefix</code>	length of the run made by fully-set blocks at the front, for the current bit
<code>stopped</code>	true once the first mixed block was found for the current bit
<code>power</code>	the answer, one entry per bit (bit 14 first)

- **Transition (bits 14 → 0), blocks $B_1, B_2, \dots$ in order:** for each B_t until the first mixed one: all-ones → `prefix += |Bt|`. First mixed block → `power = prefix + ones(Bt)`; B_t becomes `[ones][zeros]`; later blocks copied. No mixed block → `power = prefix`.
- **Base case:** one block containing all of `nums`.
- **Order:** most significant bit first (that is what lexicographic order asks for). No recursion, no table.

5. Code

Exactly as in your ZIP (LC_4059/code.cpp). Line numbers are only for reference. Note: the `!zeros.empty()` check on line 29 is always true there (we are in the `else` of line 23), so it is harmless.

```

1  #include<bits/stdc++.h>
2  using namespace std;
3  class Solution {
4  public:
5      vector<int> largestPower(vector<int>& nums) {
6          vector<vector<int>>>segments;
7          segments.push_back(nums);
8          vector<int>power;
9          for(int bit = 14;bit>=0;bit--){
10             int prefix = 0;
11             bool stopped = false;
12             vector<vector<int>>>nextSegments;
13             for (auto &seg : segments) {
14                 if (stopped){
15                     nextSegments.push_back(seg);
16                     continue;
17                 }
18                 vector<int> ones, zeros;
19                 for (int x : seg) {
20                     if ((x >> bit) & 1) ones.push_back(x);
21                     else zeros.push_back(x);
22                 }
23                 if (zeros.empty()) {
24                     prefix += seg.size();
25                     nextSegments.push_back(seg);
26                 } else {
27                     power.push_back(prefix + ones.size());
28                     if(!ones.empty()) nextSegments.push_back(ones);
29                     if(!zeros.empty()) nextSegments.push_back(zeros);
30                     stopped = true;
31                 }
32             }
33             if(!stopped) power.push_back(prefix);
34             segments = nextSegments;
35         }
36         return power;
37     }
38 };

```

6. Dry Run

nums = [7, 5, 3, 2, 1] in binary (only bits 2, 1, 0 are ever set):

number	7	5	3	2	1
bit 2	1	1	0	0	0
bit 1	1	0	1	1	0
bit 0	1	1	1	0	1

bit	blocks before	what the scan does	power	blocks after
14...3	[7 5 3 2 1]	nobody has the bit → the first block is mixed with ones = none: power = 0 + 0. Block stays whole.	0	[7 5 3 2 1]
2	[7 5 3 2 1]	ones = 7,5; zeros = 3,2,1 → mixed: power = 0 + 2. Split.	2	[7 5] [3 2 1]
1	[7 5] [3 2 1]	[7 5]: 7 has bit 1, 5 does not → mixed: power = 0 + 1. Split and stop ; [3 2 1] is copied untouched.	1	[7] [5] [3 2 1]
0	[7] [5] [3 2 1]	[7] full → prefix 1; [5] full → prefix 2; [3 2 1] has ones = 3,1 and zero = 2 → mixed: power = 2 + 2. Split.	4	[7] [5] [3 1] [2]

Result: power = [0 × 12, 2, 1, 4]. One valid permutation is **7 5 3 1 2** (block order [7] [5] [3 1] [2]). I also checked this against a brute force over all permutations, and both statement examples ([7,5] and [3,1,7]) come out as expected.

7. Complexity

	Cost	Reason
Time	$O(15 \cdot n) = O(n \cdot B)$	$B = 15$ bits; for each bit every number is either split or copied once
Space	$O(n)$	the blocks (segments + nextSegments) hold n numbers in total, at most n blocks
Recursion stack	none	plain loops

- Edge cases: $n = 1 \rightarrow \text{power}[i]$ is 1 if that bit is set, else 0. If every number has a bit, that entry is n . Zeros in nums are fine (they just never extend a run).

8. Quick Revision

Core idea:	Go bit 14 $\rightarrow$ 0. Keep an ordered list of blocks (order inside a block is free). For each bit, full blocks extend the run; the first mixed block adds its ones and is split into [ones][zeros]; later blocks are left untouched.
State:	segments (ordered blocks), prefix (run through full blocks), stopped .
Transition:	$\text{power}[\text{bit}] = \text{prefix} + \text{ones}(\text{first mixed block})$, or $\text{prefix} (= n)$ if no block is mixed.
Important observation:	Split only the first mixed block. Sorting descending / splitting everything can shorten a later run (e.g. [7,5,3,2,1]: 4 vs 3).
TC:	$O(15 \cdot n)$
SC:	$O(n)$